

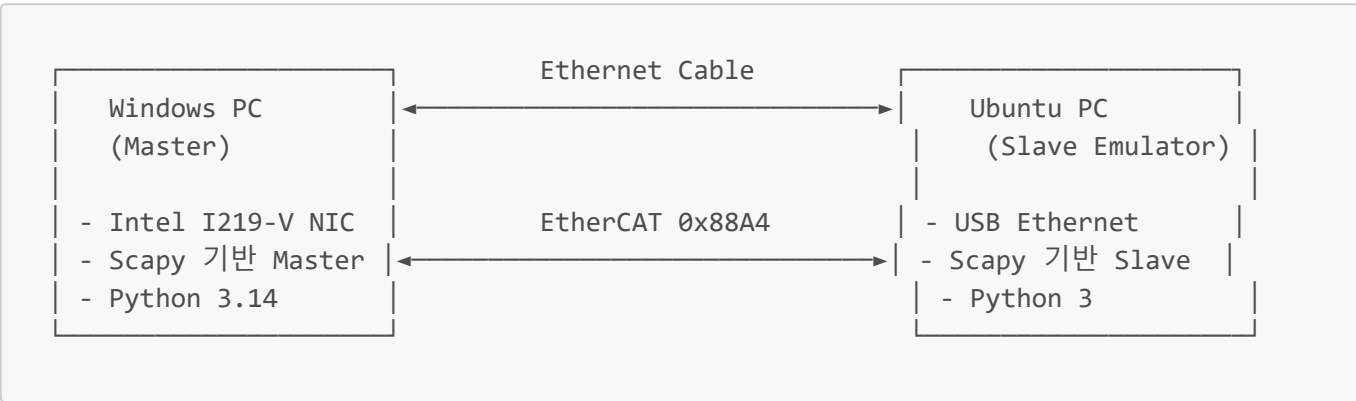
EtherCAT 시뮬레이션 프로젝트 진행 보고서

1. 프로젝트 개요

1.1 목적

실제 EtherCAT 하드웨어(ESC 칩) 없이 소프트웨어만으로 EtherCAT Master-Slave 통신을 시뮬레이션하여 프로토콜 학습 및 개발 환경 구축

1.2 시스템 구성



2. 완료된 작업 내용

2.1 Windows PC (Master 역할)

설치된 소프트웨어

소프트웨어	버전	용도
Python	3.14	스크립트 실행
Npcap	최신	Raw Ethernet 패킷 송수신
Scapy	최신	EtherCAT 프레임 생성/파싱
PySOEM	최신	(테스트용, 소프트웨어 에뮬레이터와 호환 안됨)

개발된 파일

파일명	경로	설명
ethercat_master.py	C:\todo\today\etherCat\	메인 Master 프로그램 (Slave 스캔, 상태 전이, PDO 통신)
simple_master.py	C:\todo\today\etherCat\	간단한 테스트용 Master
scan_slave.py	C:\todo\today\etherCat\	PySOEM 기반 스캔 (참고용)
test_capture.py	C:\todo\today\etherCat\	패킷 캡처 테스트

파일명	경로	설명
test_direct.py	C:\todo\today\etherCat\	직접 패킷 송수신 테스트

2.2 Ubuntu PC (Slave Emulator 역할)

설치된 소프트웨어

소프트웨어	설치 명령	용도
Python 3	기본 설치	스크립트 실행
Scapy	pip3 install scapy	EtherCAT 프레임 처리

개발된 파일

파일명	경로	설명
ethercat_slave_emulator.py	~/uttec/	최종 Slave 에뮬레이터 (v12, LRW/PDO 지원)

Ubuntu Slave Emulator의 역할

1. 가상 Slave 시뮬레이션: 3개의 Beckhoff 호환 Slave 장치 에뮬레이션
- EK1100 (EtherCAT Coupler)
 - EL1008 (8채널 디지털 입력)
 - EL2008 (8채널 디지털 출력)
2. EtherCAT 명령어 처리:
- APRD/APWR/APRW: Auto-increment 주소 기반 읽기/쓰기
 - BRD/BWR/BRW: Broadcast 읽기/쓰기
 - FPRD/FPWR/FPRW: Configured 주소 기반 읽기/쓰기
 - LRD/LWR/LRW: 논리 주소 기반 PDO 통신
3. 상태 관리: AL Status 레지스터를 통한 상태 전이 지원
- INIT (0x01) → PRE-OP (0x02) → SAFE-OP (0x04) → OP (0x08)

3. 구현된 기능 상세

3.1 EtherCAT Master (ethercat_master.py)

주요 클래스 및 메서드

```
class EtherCATMaster:
    # 초기화
    def __init__(self, adapter)          # 네트워크 어댑터 설정
```

```

# 기본 명령어
def brd_read(ado, length)      # Broadcast Read
def bwr_write(ado, data)       # Broadcast Write
def aprd_read(position, ado, len) # Auto-increment Read
def apwr_write(position, ado, data) # Auto-increment Write
def fprd_read(addr, ado, length) # Configured Address Read
def fpwr_write(addr, ado, data)  # Configured Address Write

# Slave 관리
def scan_slaves()              # Slave 스캔 (최대 63개)
def config_slaves()            # Slave 주소 할당
def read_slave_state(slave)     # 개별 Slave 상태 읽기
def write_slave_state(slave, state) # 상태 전이 요청
def transition_to_op()          # INIT → OP 자동 전이

# PDO 통신
def lrw_exchange(logical_addr, data) # Logical Read/Write
def setup_pdo_mapping()              # PDO 매핑 설정
def cyclic_exchange(output_data)     # 사이클릭 데이터 교환
def run_cyclic(duration, cycle_time) # 사이클릭 통신 실행

```

3.2 EtherCAT Slave Emulator

주요 특징

- **WKC (Working Counter) 처리**: 각 명령 처리 시 적절한 WKC 증가
- **레지스터 에뮬레이션**: ESC 레지스터 0x0000~0x2000 영역
- **Loopback 필터링**: 자신이 보낸 패킷 무시
- **Broadcast 응답**: Point-to-Point 연결에서 응답 전송

4. 사용 방법

4.1 시스템 시작 순서

Step 1: Ubuntu에서 Slave Emulator 시작

```

# Ubuntu PC에서 실행
cd ~/uttec
sudo python3 ethercat_slave_emulator.py enx00e04c680189

```

출력 예시:

```

=====
EtherCAT Slave Emulator v12 (with LRW/PDO)
=====
Interface: enx00e04c680189
Slaves: 3

```

```
[1] EK1100
[2] EL1008
[3] EL2008
=====
[HH:MM:SS.mmm] Waiting for frames... (Ctrl+C to stop)
```

Step 2: Windows에서 Master 실행

```
# Windows PC에서 실행
py -3.14 C:\todo\today\etherCat\ethercat_master.py
```

출력 예시:

```
=====
EtherCAT Master (Scapy)
=====

[HH:MM:SS.mmm] Scanning for slaves...
[HH:MM:SS.mmm] Found slave at position 1
[HH:MM:SS.mmm] Found slave at position 2
[HH:MM:SS.mmm] Found slave at position 3
[HH:MM:SS.mmm] Total slaves found: 3
...
[HH:MM:SS.mmm] All slaves in OP state!
...
[HH:MM:SS.mmm] Cycle 1: TX=000000000000, RX=080108020803
[HH:MM:SS.mmm] Cycle 100: TX=630063006300, RX=080108020803
```

4.2 네트워크 어댑터 확인

Windows

```
# 어댑터 목록 확인
py -3.14 -c "from scapy.all import IFACES; [print(f'{k}: {v.description}')] for k,v in IFACES.items()]"
```

현재 설정된 어댑터: \Device\NPF_{3EF881C7-9819-4A46-AD65-57F592C9E60E} (Intel I219-V)

Ubuntu

```
# 인터페이스 확인
ip link show
```

현재 사용 인터페이스: enx00e04c680189 (USB Ethernet)

4.3 Tailscale 원격 접속

Ubuntu PC는 Tailscale을 통해 원격 접속 가능:

```
ssh uttec@100.67.68.70
```

5. 테스트 결과

5.1 최종 테스트 결과 (2026-02-15)

항목	결과
Slave 스캔	☒ 3개 Slave 검출
주소 할당	☒ 0x1001, 0x1002, 0x1003
INIT → PRE-OP	☒ 성공
PRE-OP → SAFE-OP	☒ 성공
SAFE-OP → OP	☒ 성공
PDO 사이클릭 통신	☒ 100 사이클 에러 없음
평균 사이클 타임	50.47ms (목표: 50ms)

5.2 PDO 데이터 형식

출력 데이터 (Master → Slave)

```
TX: [Slave1 2bytes][Slave2 2bytes][Slave3 2bytes]
예: 630063006300 = 카운터 값 0x0063 (99) × 3
```

입력 데이터 (Slave → Master)

```
RX: [Slave1 2bytes][Slave2 2bytes][Slave3 2bytes]
예: 080108020803
    08 01 = state=0x08(OP), position=1
    08 02 = state=0x08(OP), position=2
    08 03 = state=0x08(OP), position=3
```

6. 향후 진행 계획

6.1 단기 과제 (1-2주)

1) 실제 I/O 데이터 시뮬레이션

- EL1008 (디지털 입력): 가상 입력 신호 생성
- EL2008 (디지털 출력): 출력 상태 로깅
- 타이머 기반 입력 값 변화 구현

2) CoE (CANopen over EtherCAT) 구현

- SDO (Service Data Object) 통신 추가
- Object Dictionary 구현
- 파라미터 읽기/쓰기 기능

3) SyncManager 및 FMMU 구현

- SM2/SM3 (PDO 영역) 매핑
- FMMU를 통한 논리 주소 매핑

6.2 중기 과제 (1-2개월)

1) DC (Distributed Clock) 동기화

- DC 레지스터 에뮬레이션
- 시간 동기화 프로토콜 구현

2) ESI/XML 파일 기반 Slave 생성

- ESI (EtherCAT Slave Information) 파서
- 동적 Slave 구성

3) GUI 모니터링 도구

- 실시간 상태 모니터링
- PDO 데이터 시각화
- 네트워크 토폴로지 표시

6.3 장기 과제 (2-3개월)

1) 실제 하드웨어 연동 테스트

- ESC 칩 (LAN9252 등) 구매 및 테스트
- 실제 Beckhoff Slave 연동

2) PySOEM 호환성 개선

- PySOEM이 에뮬레이터와 동작하도록 타이밍 조정
- 또는 자체 Master 라이브러리 완성

3) 산업용 응용 개발

- 모션 컨트롤 프로토콜 (CiA402)
- PLC 연동 인터페이스

7. 기술적 참고사항

7.1 EtherCAT 프로토콜 요약

프레임 구조

Ethernet Hdr (14 bytes)	EtherCAT Hdr (2 bytes)	Datagrams (가변 길이)
Dst MAC Src MAC Type=0x88A4	Length + Type	[DG1][DG2]...[DGn]

Datagram 구조

CMD	IDX	ADP	ADO	LEN+FLG	IRQ	DATA	WKC
1B	1B	2B	2B	2B	2B	nB	2B

7.2 주요 레지스터 주소

주소	이름	설명
0x0000	Type	ESC 타입
0x0001	Revision	ESC 리비전
0x0010	Station Address	설정된 주소
0x0110	DL Status	데이터 링크 상태
0x0120	AL Control	AL 상태 제어
0x0130	AL Status	AL 현재 상태
0x0134	AL Status Code	에러 코드

7.3 WKC (Working Counter) 규칙

명령	성공 시 WKC 증가
----	-------------

명령	성공 시 WKC 증가
Read (APRD, FPRD, BRD, LRD)	+1
Write (APWR, FPWR, BWR, LWR)	+1
Read/Write (APRW, FPRW, BRW, LRW)	+3

8. 문제 해결 가이드

8.1 Slave를 찾지 못하는 경우

1. Ubuntu 에뮬레이터 실행 확인

```
ps aux | grep ethercat
```

2. 네트워크 케이블 연결 확인

- Ethernet 케이블이 직접 연결되어 있는지 확인
- 스위치/허브를 거치면 안 됨 (EtherCAT은 Point-to-Point)

3. 인터페이스 상태 확인

```
# Ubuntu
ip link show enx00e04c680189
# 상태가 UP인지 확인
```

8.2 PDO 통신 실패 (WKC=0)

1. Slave가 OP 상태인지 확인

- LRW 명령은 OP 상태에서만 응답

2. 에뮬레이터 버전 확인

- v12 이상 사용 (LRW 지원)

8.3 응답 지연 문제

1. 사이클 타임 조정

- `run_cyclic()` 함수의 `cycle_time` 파라미터 증가
- 현재: 50ms, 필요시 100ms로 조정

9. 파일 구조 요약


```
C:\todo\today\etherCat\  
├─ ethercat_master.py      # 메인 Master 프로그램  
├─ simple_master.py       # 간단한 테스트 Master  
├─ scan_slave.py          # PySOEM 스캔 (참고용)  
├─ test_capture.py        # 패킷 캡처 테스트  
├─ test_direct.py         # 직접 송수신 테스트  
├─ emulator_v12.py        # 에뮬레이터 복사본  
├─ prompt.txt             # 프로젝트 프롬프트  
└─ next/  
    └─ 향후진행보고서.md   # 이 문서  
  
Ubuntu ~/uttec/  
└─ ethercat_slave_emulator.py # 최종 Slave 에뮬레이터 (v12)
```

작성일: 2026-02-15 **작성자:** Claude Code Assistant **버전:** 1.0